School of Future Tech

Case Study Report

on

CleanSweep: On-Demand Home Cleaning

by

Kushal

BTech CSE — 2025–29 Batch, Semester 2

ITM Skills University, Navi Mumbai

ROLL NUMBER :- 150096725025

Index

#	Section
1.	Introduction to the Case Study
2.	Problem Statement / Case Background (Abstract)
3.	Problem Statement / Case Study Design
4.	Methods & Algorithms / Technology Applied
5.	Implementation Details and Snapshots
6.	Results and Conclusion
7.	References

1. Introduction to the Case Study

The on-demand home services sector in urban India has seen explosive growth driven by nuclear households, dual-income families, and smartphone penetration. Despite this, most existing platforms lack pricing transparency, offer poor service customisation, and deliver a subpar booking experience — especially on the web.

CleanSweep is a front-end web application designed as a case study to explore how a modern, responsive, user-centric booking interface can be built entirely in vanilla HTML5, CSS3, and JavaScript — without any frameworks or build tools. It simulates a real product: a home cleaning booking platform where users browse service categories, build a custom task checklist, calculate costs dynamically, and proceed through a checkout flow.

The case study covers the full frontend development lifecycle, from UI design decisions and component architecture to JavaScript state management and responsive layout. It also explores future product directions such as authenticated accounts, booking history, and an admin dashboard.

2. Problem Statement / Case Background (Abstract)

Background

Urban households increasingly rely on professional cleaners and home help, yet booking such services remains fragmented and opaque. Existing platforms often present fixed-package pricing with no granular control, no real-time cost feedback, and cluttered interfaces not optimised for mobile. First-time users frequently abandon bookings due to a lack of trust signals — such as cleaner background checks, eco-friendly product options, and transparent pricing.

Abstract

This case study presents the design and implementation of CleanSweep, an on-demand home cleaning booking web app. The core question it addresses is: How can a user book professional home cleaners with specific task checklists, accurate price estimation, and a seamless checkout experience using only client-side web technologies?

The solution is a multi-page vanilla JS frontend with a dynamic price estimator, service-category browsing, and a sessionStorage-based checkout pipeline. The project demonstrates that a polished, production-quality product experience can be achieved without heavy frameworks by using well-structured HTML semantics, modular CSS with variables, and clean JS module separation.

3. Case Study Design

CleanSweep is architected as a multi-page application (MPA) with clearly separated concerns across HTML views, CSS stylesheets, and JavaScript modules. The system is designed around five core user journeys:

- Service Discovery: The user lands on the homepage and browses available cleaning categories (General Cleaning, Dishwashing, Laundry, Kitchen Prep, Bathroom Cleaning).
- Task Customisation: On the Services page, the user selects tasks and durations, with the price estimator updating totals in real time.
- Checkout Flow: Selected services are persisted via sessionStorage and surfaced on the Checkout page for review and confirmation.
- Trust Building: FAQ, testimonials, cleaner background check badges, and eco-friendly product indicators are surfaced across pages.
- Navigation & Feedback: A persistent navbar, smooth scroll, and micro-animations guide the user experience.

Page Structure

File / Folder	Description
index.html	Root entry point and redirect handler
html/index.html	Main landing page — hero, service grid, how-it-works, testimonials, FAQs
html/services.html	Service checklist and interactive task selector
html/checkout.html	Booking summary and confirmation form
html/dashboard.html	User dashboard view (Phase 2)

Visual Design Decisions

The visual identity is inspired by premium on-demand service apps. It uses a maroon (#3C172A) and pink (#E91E7B) palette, Sofia Sans Extra Condensed for display headings, and Satoshi for body text. Cards use soft shadows and border-radius for a modern feel. All layout is achieved with CSS Flexbox and Grid — no external UI libraries.

4. Methods & Algorithms / Technology Applied

Core Technologies

- **HTML5:** HTML5:
  - Semantic markup (header, main, section, article, nav) for accessible, SEO-friendly structure.
  - Session Storage API for cross-page cart state.
- **CSS3:** CSS3:
  - Custom properties (variables) as a design token system for colours, fonts, spacing.
  - Flexbox and CSS Grid for responsive layouts.
  - Keyframe animations for micro-interactions. Media queries for mobile breakpoints.
- **Vanilla JavaScript (ES6+):** Vanilla JavaScript (ES6+):
  - Module-based script separation (priceestimator.js, auth.js, etc.).
  - DOM manipulation for dynamic rendering of service cards and cart.
  - Event delegation for performance. sessionStorage for cart persistence.

Key Algorithm: Dynamic Price Estimator

The price estimator (priceestimator.js) is the technical centrepiece. It implements the following logic:

- Each service task has a base hourly rate defined in a JS data object.
- When a user selects a task and duration: Cost = Hourly Rate × Duration Hours.
- Multiple tasks accumulate in a cart array stored in memory and synced to sessionStorage.
- On any cart mutation, total price and estimated time are recalculated and re-rendered in real time.
- Square footage multipliers apply as a modifier to base rate for area-dependent tasks.
- Weekly recurring discount (e.g. 15% off) computed and shown as a conditional line item.

Technology Stack

Layer	Technology	Purpose
Structure	HTML5	Page layout and semantics
Styling	CSS3 + Custom Properties	Design system and responsive UI
Logic	Vanilla JavaScript ES6+	Cart, state, DOM interactions
State	sessionStorage API	Cross-page cart persistence
Assets	SVGs & PNGs	Icons, illustrations, brand assets
Hosting	Static (any server)	No build tools required

5. Implementation Details and Snapshots

Project Structure (github.com/kushal-sw/clean-sweep)

```
clean-sweep/
├── index.html                ← Root entry point
├── html/                     ← All HTML views
├── css/                      ← Global + page-specific stylesheets
├── js/                       ← JS modules (priceestimator.js, auth.js …)
└── images/                   ← SVGs, PNGs, icons
```

Key Implementation Highlights

- CSS Custom Properties System: All colours, font families, and spacing scales are defined as CSS variables on :root — a single source of truth for the design system.
- Modular JavaScript Architecture: Logic is split across purpose-specific .js files. The price estimator module is self-contained, reducing coupling and making testing easier.
- sessionStorage Cart Persistence: Cart state is serialised to JSON on every mutation. The checkout page deserialises and renders it — seamless cross-page flow with no backend required.
- Responsive Layout Without Frameworks: CSS Grid for two-dimensional structures and Flexbox for one-dimensional flows. Media queries handle 768px and 480px breakpoints.
- Micro-Animations & UX Polish: CSS keyframe animations on card hover, smooth scroll, and transition properties on buttons contribute to the premium feel.

Application Snapshots

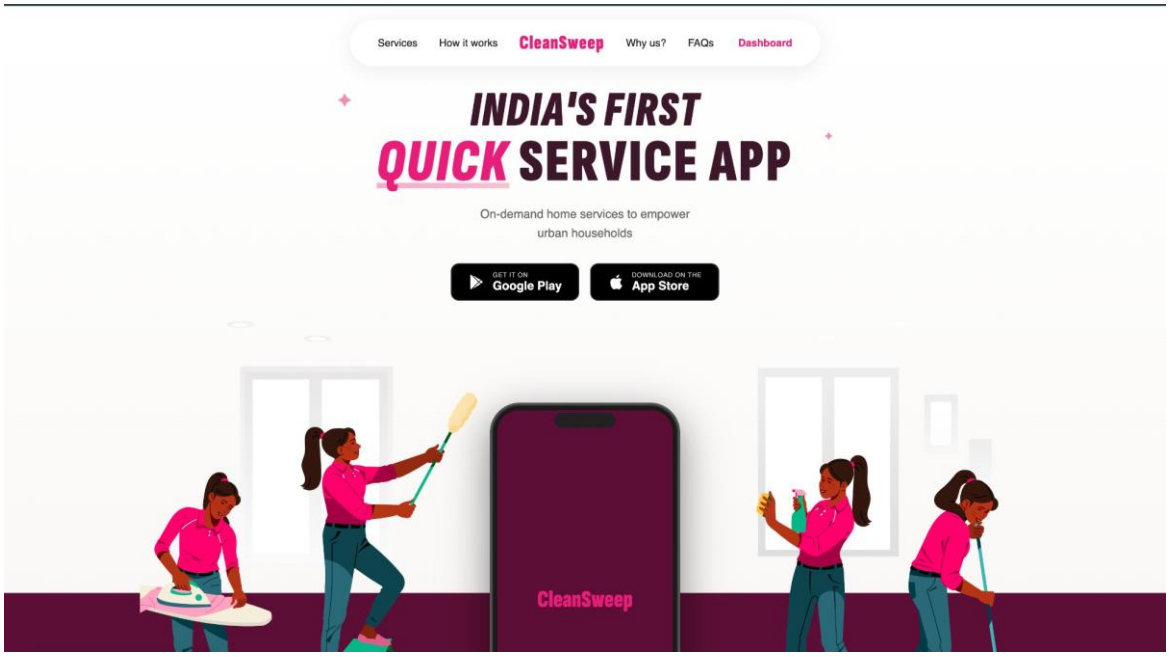

Figure 1 — Homepage Hero: India's First Quick Service App, with service navigation and illustrated cleaners

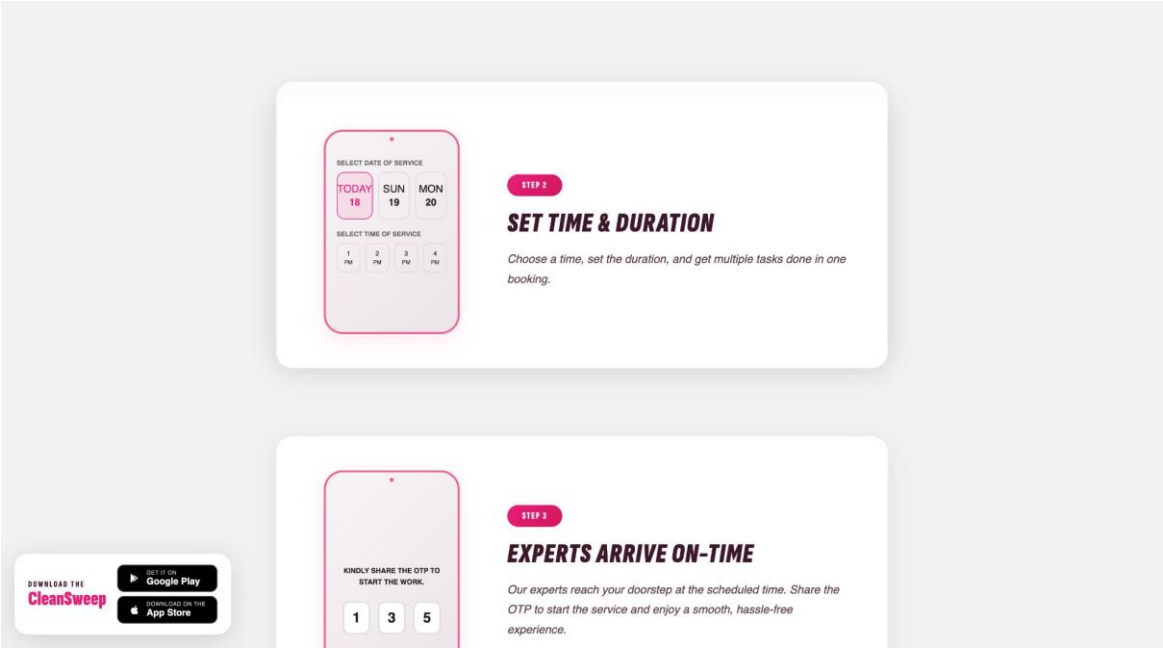

Figure 2 — How It Works: Step-by-step booking flow — Set Time & Duration, Experts Arrive On-Time

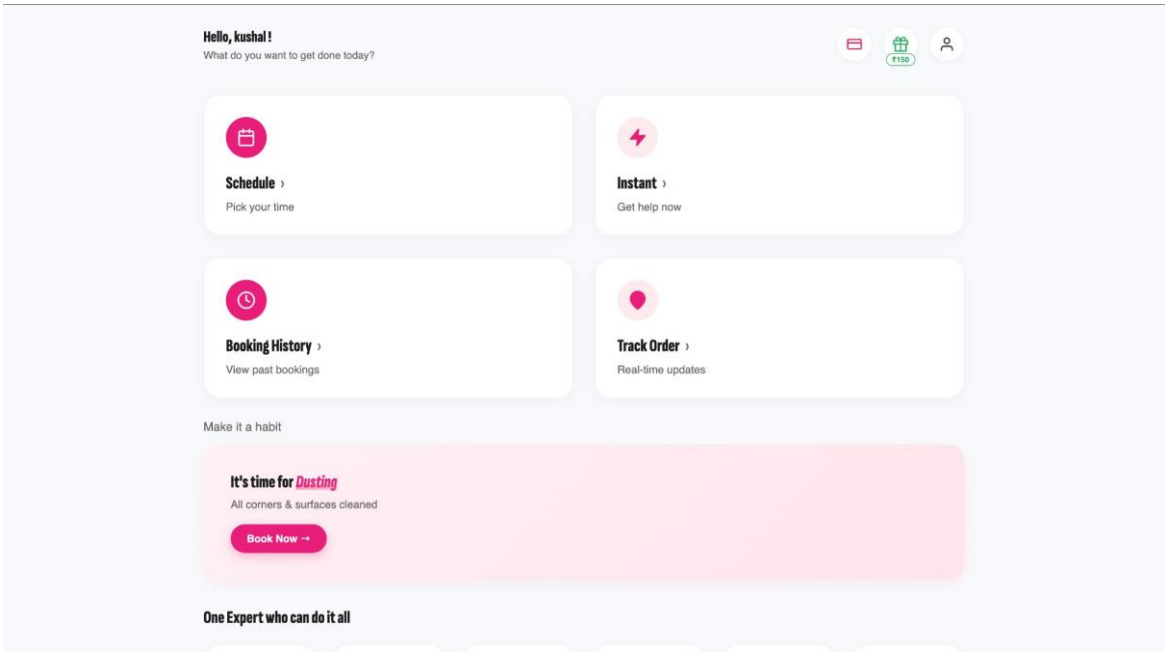

Figure 3 — User Dashboard: Schedule, Instant, Booking History, Track Order, and personalised habit suggestions

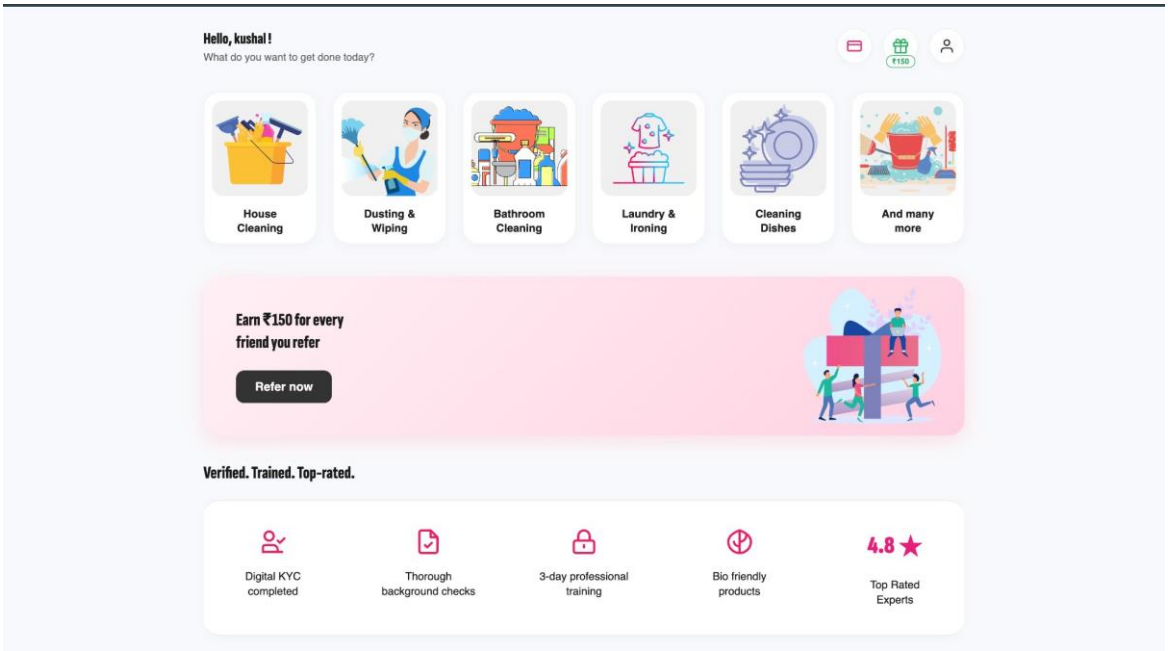

Figure 4 — Dashboard Service Grid: Service categories, referral banner, and Verified/Trained/Top-rated trust badges

6. Results and Conclusion

Findings

- A complete, multi-page web app with professional UX can be built entirely in vanilla HTML, CSS, and JS — framework adoption is a choice, not a necessity.
- CSS custom properties proved to be a powerful design system primitive, eliminating the need for a utility-class framework while keeping the codebase DRY.
- The dynamic price estimator demonstrated that real-time reactive UIs can be cleanly built without React or Vue by using well-structured event listeners and a single state object.
- sessionStorage as a cross-page state mechanism worked effectively for the cart, validating that simple apps can avoid complex state management solutions.
- Separating CSS into global and page-specific files, and JS into feature modules, significantly improved maintainability — a pattern that scales well in larger teams.
- The project achieved high visual fidelity to the target aesthetic using only custom CSS, confirming that design quality depends more on intentional decisions than on component libraries.

Conclusion

CleanSweep successfully demonstrates a complete frontend product pipeline: from problem identification (fragmented home service booking) to a polished, deployable web interface. The project integrates responsive UI design, modular JavaScript architecture, real-time price calculation, and cross-page state management into a cohesive, user-centred application.

Beyond technical implementation, the case study illustrates real product thinking — defining service categories, pricing strategy (square footage pricing, recurring discounts), and trust-building features (eco-friendly options, background checks) that mirror real startup decisions in the home services space.

The codebase at github.com/kushal-sw/clean-sweep serves as a strong portfolio piece demonstrating frontend fundamentals, UI/UX sensibility, and the ability to ship a complete product independently. Phase 2 development — including authentication, booking history, admin dashboard, and toast notifications — is planned for continued iteration.

7. References

1. CleanSweep GitHub Repository — github.com/kushal-sw/clean-sweep (Kushal, 2025)
2. MDN Web Docs — HTML5, CSS3 Custom Properties, sessionStorage, DOM API. developer.mozilla.org
3. CSS Tricks — A Complete Guide to Flexbox and CSS Grid. css-tricks.com
4. JavaScript.info — Modern JavaScript fundamentals, ES6 modules, and DOM manipulation. javascript.info
5. Google Fonts — Sofia Sans Extra Condensed typeface. fonts.google.com
6. Fontshare — Satoshi typeface by Indian Type Foundry. fontshare.com
7. Snabbit (design reference) — On-demand home services platform. snabbit.com
8. Can I Use — Browser compatibility reference for CSS and JS features. caniuse.com